Corso di Architettura degli Elaboratori e Laboratorio (F-N)

Circuiti aritmetici

Massimo Orazio Spata

Dipartimento di Matematica e Informatica



UNIVERSITÀ
degli STUDI
di CATANIA

Addizionatore ad 1 bit

Per **SOMMARE** numeri binari ad 1 bit:

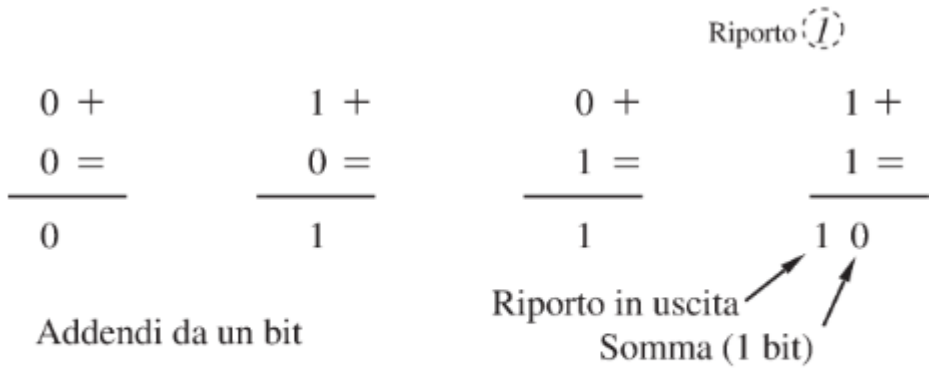


Figura 1.4 - Addizione di numeri a un bit

Il **RIPORTO IN USCITA** della cifre precedente viene assegnato come **RIPORTO IN ENTRATA** alla successiva

Addizionatore ad 1 bit

- Un addizionatore tra due singoli bit può essere espresso da 2 funzioni logiche a tre ingressi (i due bit da sommare più il riporto in ingresso):
  - La prima calcola la somma tra i bit ed il riporto in ingresso
  - La seconda calcola il riporto in uscita
- Dalla tabella di verità si ricavano le espressioni logiche per somma e riporto in uscita

x_i	y_i	Riporto in ingresso c_i	Somma s_i	Riporto in uscita c_{i+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$s_i = \bar{x}_i\bar{y}_i r_i + \bar{x}_i y_i \bar{c}_i + x_i \bar{y}_i \bar{c}_i + x_i y_i c_i = x_i \oplus y_i \oplus c_i$$

$$c_{i+1} = x_i c_i + y_i c_i + x_i y_i$$

Addizionatore ad 1 bit: esempio

Qui si introduce il concetto fondamentale: **sommare due bit** non è sufficiente, perché dobbiamo anche considerare il **riporto** dalla cifra precedente.

Esempio:

- 1 + 1 = 0 con riporto 1

Questo riporto diventa **ingresso** per il bit successivo. È lo stesso principio dell'addizione "a mano" in base 10.

Un addizionatore a 1 bit ha:

- **3 ingressi**:
  - x, y (bit da sommare)
  - c_i (riporto in ingresso)
- **2 uscite**:
  - s_i (somma)
  - c_{i+1} (riporto in uscita)

Dalla tabella di verità:

- la **somma** è una XOR tra i tre ingressi
- il **riporto** vale 1 quando almeno due ingressi sono 1

Messaggio importante: ogni operazione aritmetica può essere tradotta in **equazioni booleane**.

Addizionatore ad 1 bit: esempio

Abbiamo un **addizionatore ad 1 bit** con:
x = 1
y = 0
c_i = 1 (riporto in ingresso)
Stiamo quindi sommando **tre bit**.

Passo 1 –
Facciamo la somma in decimale, non come circuito.
Sommiamo i valori numerici: 1 + 0 + 1 = 2
Fin qui tutto chiaro.

Passo 2 –
Scriviamo 2 in binario
In binario: 2 = 10
Questo è il punto chiave.
Il risultato non è un solo bit, ma due bit:

- il bit meno significativo è 0
- il bit più significativo è 1

Passo 3 –
Cosa può fare un addizionatore ad 1 bit?
Un addizionatore ad 1 bit non può restituire due bit come somma.
Per convenzione: il bit meno significativo diventa la somma s_i il bit più significativo diventa il riporto c_{i+1}
Quindi:

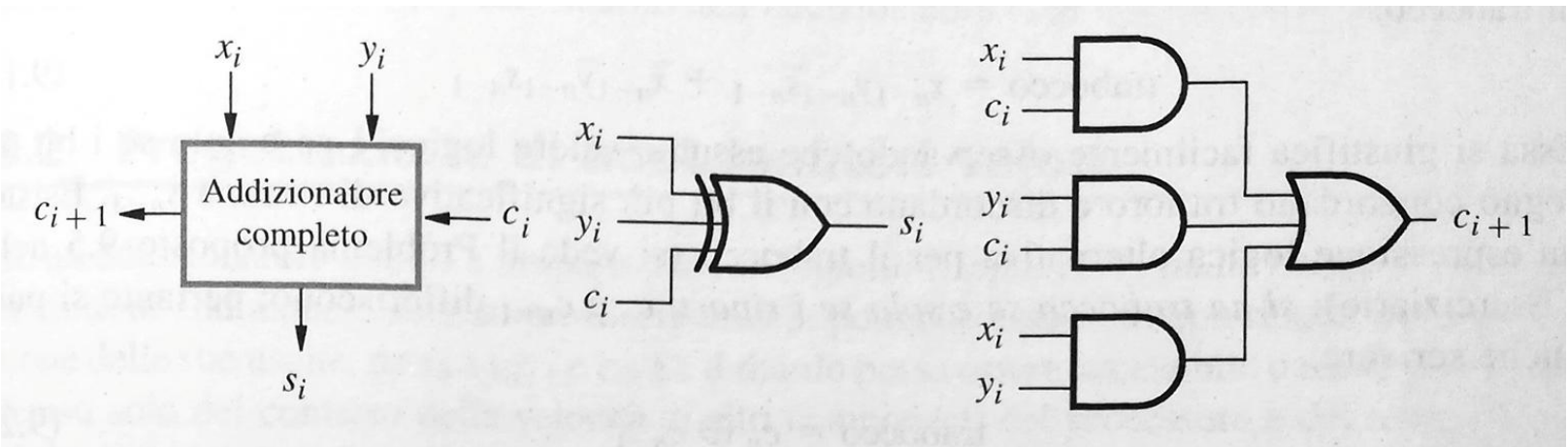
1 0
↑ ↑
| └── somma (s_i) = 0
└──── riporto (c_{i+1}) = 1

Passo 4 –
Risultato finale del circuito
somma = 0
riporto = 1

Addizionatore completo (full adder)

- Unendo assieme in un singolo circuito le reti logiche per le funzioni di somma e riporto in uscita si ottiene l'addizionatore completo

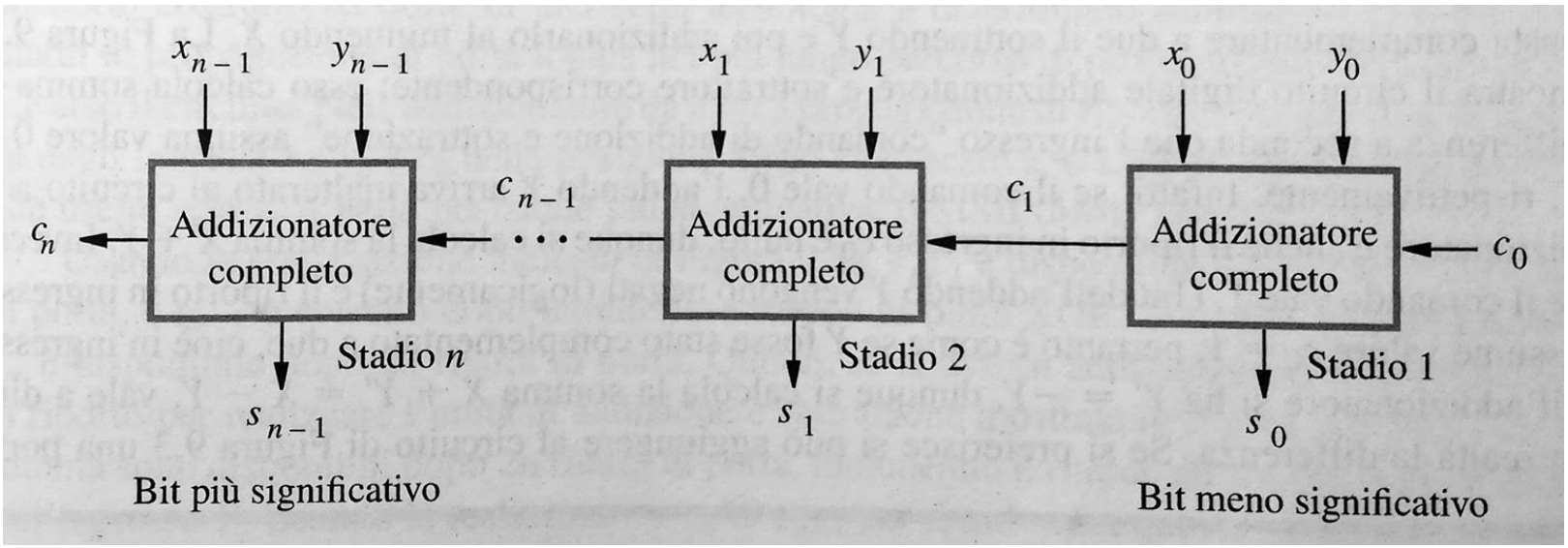
- L'addizionatore completo prende in ingresso i due bit da sommare e il riporto in entrata e rende in uscita somma e riporto in uscita



Addizionatore a propagazione di riporto

- Collegando una catena di n addizionatori completi in modo da propagare il riporto si ottiene un circuito in grado di sommare numeri binari di n bit

- Tale circuito è chiamato addizionatore a propagazione di riporto (ripple carry adder)



Addizionatore a propagazione di riporto: esempio

Collegando più full adder si sommano numeri con più bit.

Esempio

Somma di due numeri a 4 bit:

```
0110   (6) +
0011   (3)
------
1001   (9)
```

Il riporto passa dal bit meno significativo al più significativo.

Trabocco

- L'addizione di due numeri in complemento a due corrisponde alla somma di due numeri binari naturali senza contare il riporto in uscita

- La sottrazione corrisponde ad un addizione complementando a due il sottraendo

- Bisogna però garantire che non avvenga trabocco

- Il calcolo del trabocco può essere espresso da una delle seguenti espressioni logiche:

$$trabocco = x_{n-1} y_{n-1} \bar{s}_{n-1} + \bar{x}_{n-1} \bar{y}_{n-1} s_{n-1}$$

$$trabocco = c_n \oplus c_{n-1}$$

Trabocco: esempio

Il trabocco avviene quando il risultato non è rappresentabile con i bit disponibili.

Esempio

Con 4 bit in complemento a due: (2^4 = 16 => da -8 a +7 con 1 bit per il segno)

- massimo positivo = +7
- 5 + 5 = 10 → overflow

- Il risultato diventa negativo, ma **matematicamente è sbagliato**.

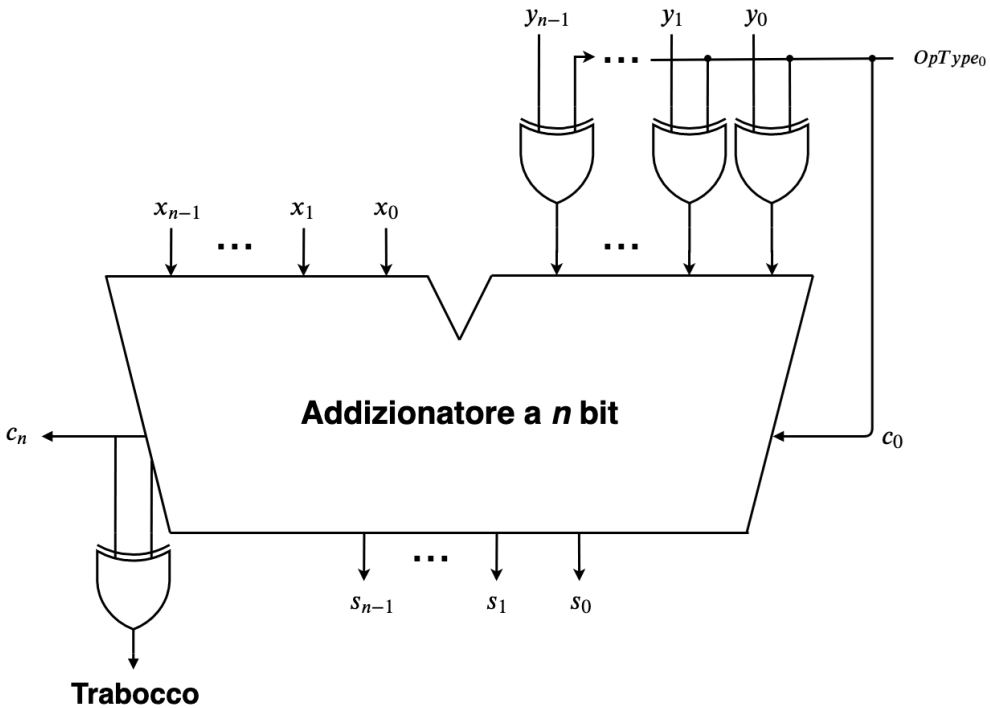
Addizionatore algebrico a n bit

- Una unità logica per addizione e sottrazione può essere ottenuta usando un addizionatore a propagazione di riporto

- Si usa il bit $OpType_0$ per complementare a due il sottraendo in caso di sottrazione

- Nel caso $OpType_0 = 1$ si avrà un riporto in ingresso al bit meno significativo e il secondo addendo verrà complementato attraverso una catena di porte xor parallele ($y_n \oplus OpType_0$)

- Il trabocco viene calcolato come $c_n \oplus c_{n-1}$



Addizionatore algebrico a n bit: esempio

Lo stesso circuito può fare addizione o sottrazione.

Esempio
Per fare 7 − 3:
•si fa il complemento a due di 3
•poi si somma: 7 + (−3)

Il circuito **non sa** che sta sottraendo: sta solo sommando.
Significa questo: 👉 non servono due circuiti diversi (uno per + e uno per −). 👉 lo stesso addizionatore può fare entrambe le operazioni, se prepariamo bene gli ingressi.
Il trucco è usare il complemento a due.

Il ruolo del bit $OpType_0$ “Si usa il bit $OpType_0$ per complementare a due il sottraendo in caso di sottrazione”

$OpType_0 = 0 \rightarrow$ addizione
$OpType_0 = 1 \rightarrow$ sottrazione

Questo bit controlla tutto il comportamento del circuito.

Addizionatore algebrico a n bit: esempio

3. Come funziona la sottrazione nel circuito?

Nel caso $OpType_o = 1$ si avrà un riporto in ingresso al bit meno significativo e il secondo addendo verrà complementato
Ricorda la regola matematica: $A-B=A+$(complemento a due di B)

Il complemento a due di B si ottiene così: inverti tutti i bit di B aggiungi 1
Nel circuito: l'inversione dei bit è fatta con le porte XOR il +1 è fatto mettendo $OpType_o = 1$ come riporto iniziale.

4. Le porte XOR parallele ($y_n \oplus OpType_o$)
Guarda la parte alta dello schema: ogni bit y_i passa in una XOR con $OpType_o$.

$OpType_o$	y_i	$y_i \oplus OpType_o$
0	y_i	y_i (non cambia)
1	y_i	$\neg y_i$ (invertito)

👉 Quindi: addizione → y passa invariato
sottrazione → y viene complementato

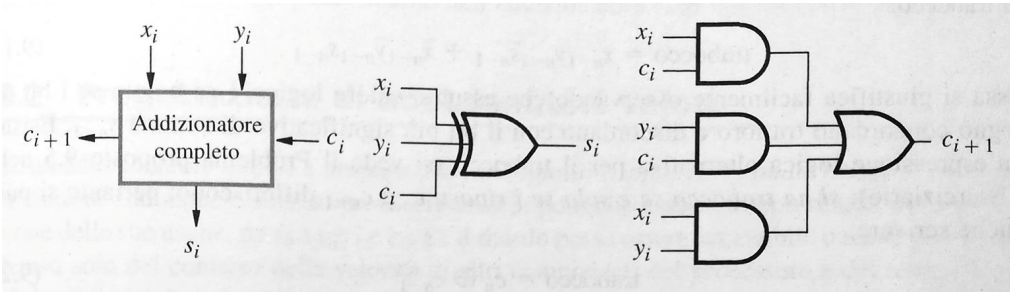
Ritardi ripple carry adder

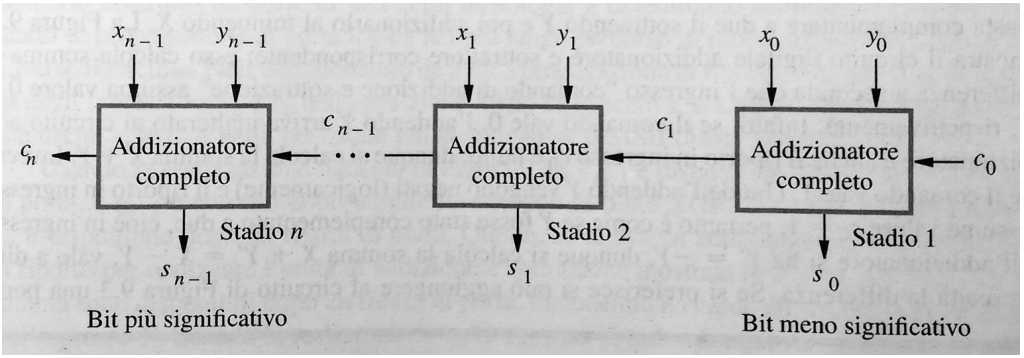
- Il ritardo totale di un circuito dipende dal ritardo del percorso più lento

- Assumiamo che ogni porta logica semplice introduce un ritardo fisso

- Un **Full Adder** genera s_i dopo **1 ritardo di porta**, mentre c_{i+1} dopo **2 ritardi di porta**

- Quindi in un **Ripple Carry Adder** a **n bit** il riporto c_n viene generato in **2n ritardi di porta**, mentre l'ultimo bit risultato s_{n-1} viene generato dopo **2n - 1 ritardi di porta**





Funzioni di generazione e propagazione

- Il risultato di ciascun Full Adder dipende dal riporto calcolato dal Full Adder nella posizione anteriore:

$$s_i = x_i \oplus y_i \oplus c_i \qquad c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

- Fattorizzando l'equazione del riporto si ottiene:

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i = x_i y_i + (x_i + y_i) c_i = G_i + P_i c_i$$

$$\mathbf{G_i = x_i y_i} \qquad \mathbf{P_i = x_i + y_i}$$

- **G_i (funzione di generazione) e P_i (funzione di propagazione)** dipendono solo dagli ingressi x_i e y_i e possono essere calcolati tutti in parallelo in **1 ritardo di porta**

Parallelizzare il calcolo del riporto

È possibile calcolare i tutti ritardi c_i solo in funzione degli addendi X, Y e del riporto in ingresso c_0?

- Espandendo iterativamente c_i fino ad arrivare a c_0 si ottiene un'equazione per **c_{i+1} in funzione solo dei vari P, G e di c_0**:

$$c_{i+1} = G_i + P_i c_i = G_i + P_i(G_{i-1} + P_{i-1}c_{i-1}) = G_i + P_iG_{i-1} + P_iP_{i-1}c_{i-1}$$

$$c_{i+1} = G_i + P_iG_{i-1} + P_iP_{i-1}G_{i-2} + \ldots + P_iP_{i-1}\ldots P_1G_0 + P_iP_{i-1}\ldots P_0c_0$$

- In questo modo ciascun riporto ci può essere **calcolato in parallelo dopo 2 ritardi di porta**

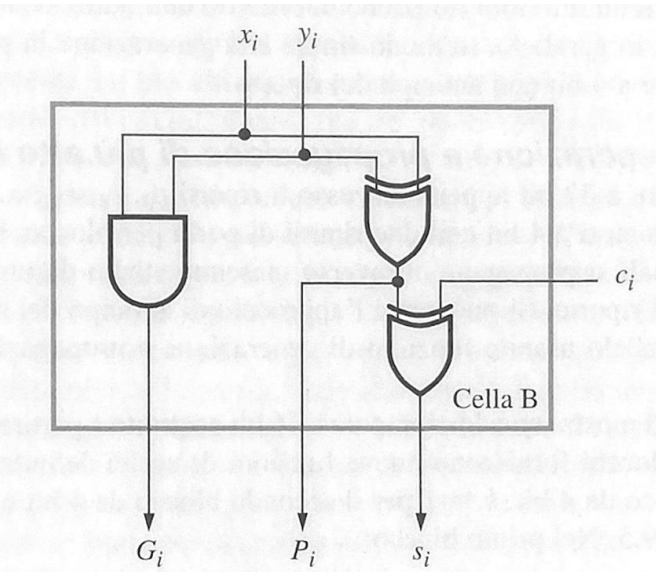
Cella di stadio da un bit

- Si può modificare la cella di sommatore a 1 bit per dare in uscita anche le funzioni $G_i = x_i y_i$ e $P_i = x_i + y_i$

- G_i si ottiene con **una porta AND con ingressi x_i e y_i**

- P_i si ottiene con **una porta XOR con ingressi x_i e y_i**

- s_i si ottiene con **due porte XOR annidate con ingressi x_i, y_i e c_i**



Addizionatore con anticipo di riporto a 4 bit

- Usando 4 celle da un bit è possibile realizzare un **Addizionatore con anticipo di riporto** a 4 bit

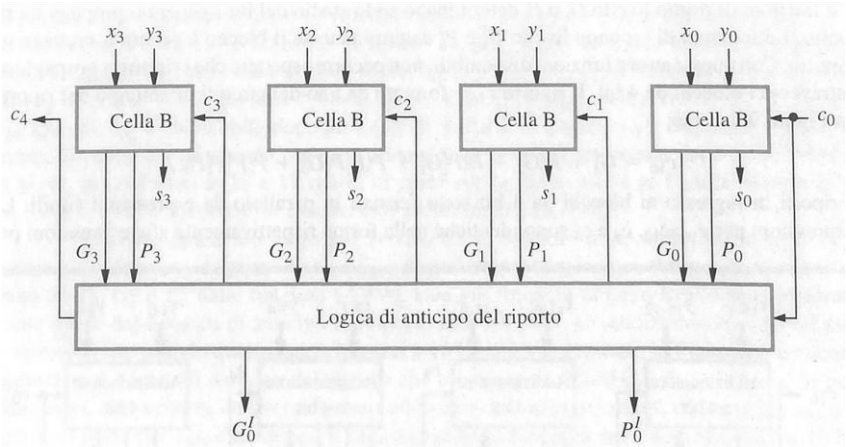
- Il blocco "**Logica di anticipo del riporto**" genera i riporti come segue:

$$c_1 = G_0 + P_0c_0$$

$$c_2 = G_1 + P_1G_0 + P_1P_0c_0$$

$$c_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0c_0$$

$$c_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0c_0$$



- **c4** viene generato dopo **3 ritardi di porta**, mentre il **risultato S** dopo **4 ritardi di porta**

Addizionatore con anticipo a 2 livelli

- Con addizionatori con anticipo di riporto a più di 4 celle si incorre in valori di **fan-in troppo elevati** nelle porte di generazione dei riporti

- Una soluzione è replicare l'idea di anticipo di riporto su **più livelli**

- In un addizionatore a 4 bit si ha:

$$c_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0c_0 = \mathbf{G^I_k} + \mathbf{P^I_k}c_0$$

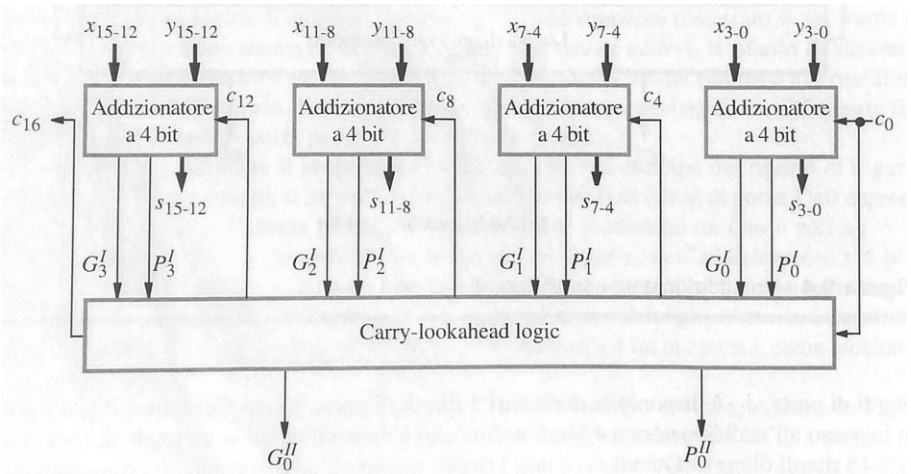
$$\mathbf{G^I_k} = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0$$

$$\mathbf{P^I_k} = P_3P_2P_1P_0$$

- Per ottenere un **addizionatore da 16 bit con anticipo di riporto**, si possono collegare **4 addizionatori a 4 bit** con un blocco di anticipo di riporto che riceve in ingresso i vari G^I_k e P^I_k e genera in parallelo c_4, c_8, c_{12} e c_{16}

Addizionatore con anticipo di riporto a 16 bit

- Usando 4 addizionatori a 4 bit è possibile realizzare un **Addizionatore con anticipo di riporto** a 16 bit su due livelli



- Il blocco "**Logica di anticipo del riporto**" genera i riporti come segue:

$$c_4 = G^I_0 + P^I_0 c_0$$

$$c_8 = G^I_1 + P^I_1 G^I_0 + P^I_1 P^I_0 c_0$$

$$c_{12} = G^I_2 + P^I_2 G^I_1 + P^I_2 P^I_1 G^I_0 + P^I_2 P^I_1 P^I_0 c_0$$

$$c_{16} = G^I_3 + P^I_3 G^I_2 + P^I_3 P^I_2 G^I_1 + P^I_3 P^I_2 P^I_1 G^I_0 + P^I_3 P^I_2 P^I_1 P^I_0 c_0$$

- **c16** viene generato dopo **5 ritardi di porta**, mentre il **risultato S** dopo **8 ritardi di porta**

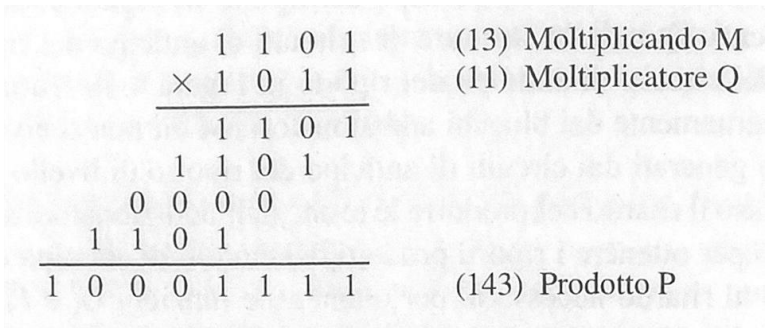
Moltiplicazione numeri senza segno

- Il metodo imparato a scuola per eseguire la moltiplicazione di due numeri vale anche per i numeri in formato binario

- Si moltiplica ciascuna cifra del moltiplicatore per il moltiplicando e si sommano i risultati fatti scorrere di una posizione ogni cifra

- Nel caso binario il Moltiplicando è moltiplicato solo per 0 o per 1

- Il prodotto di due numeri da n cifre è composto da 2n cifre

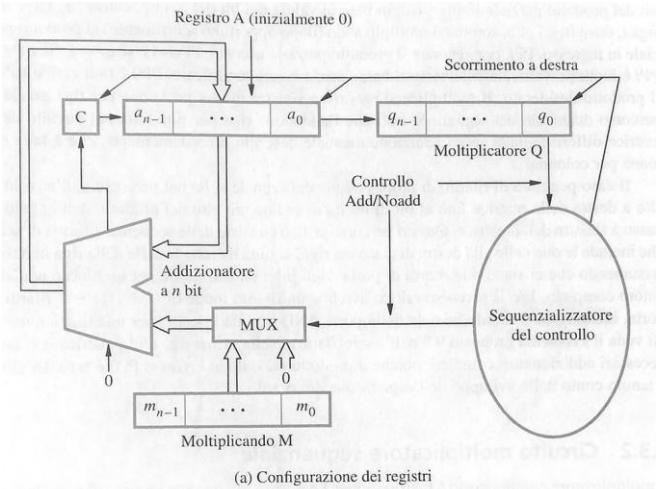


Circuito moltiplicatore sequenziale

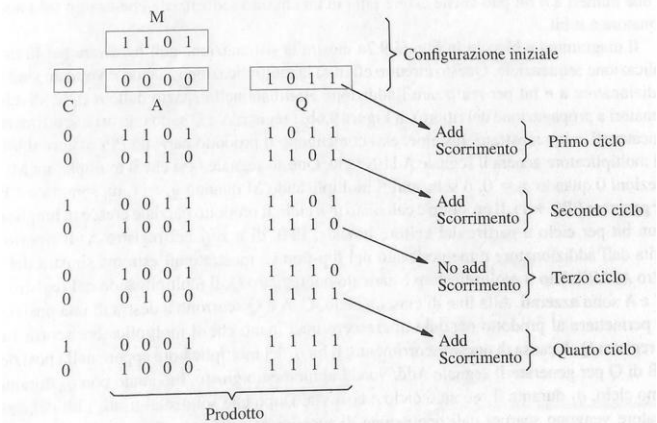
- La moltiplicazione di numeri senza segno può essere realizzata sequenzialmente usando solo **un Addizionatore a n bit e due registri a scorrimento**

- Ad ogni cilclo, l'Addizionatore effettua la somma tra il Moltiplicando (o un array di zeri) e un prodotto parziale fatto scorrere a destra di una posizione

- I due registri a scorrimento contengono:
  - **Inizialmente:** Registro A = 0 e Registro Q = Moltiplicatore
  - **Alla fine:** Registro A || Registro Q = Prodotto



(a) Configurazione dei registri



(b) Esempio di moltiplicazione

Circuito moltiplicatore sequenziale

- Algoritmo:

  A = 0

  Q = Moltiplicatore

  M = Moltiplicando

  Per n cicli:

  se q_0 = 1:

  A = A + M

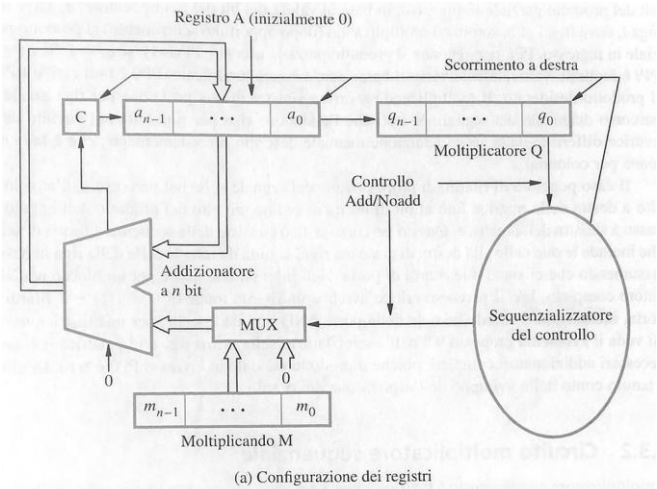
  altrimenti:

  A = A + 0

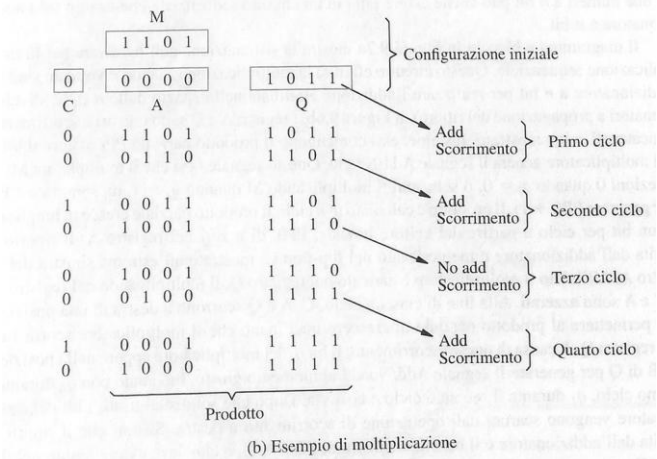
  c = riporto

  c || A || Q scorrono verso destra di una posizione

- Dopo n cicli i due registri **A || Q** concatenati conterranno il **Prodotto finale**



(a) Configurazione dei registri

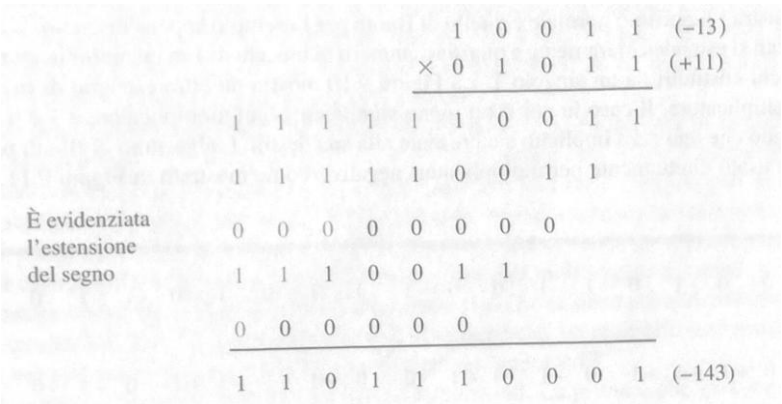


(b) Esempio di moltiplicazione

Moltiplicazione di numeri con segno

- Per moltiplicare numeri con segno in complemento a 2 bisogna apportare delle modifiche all'algoritmo di moltiplicazione

- Soluzione diretta:
  - **Moltiplicatore positivo:** si procede normalmente
  - **Moltiplicando positivo:** si procede normalmente
  - **Moltiplicando negativo:** Bisogna estendere il segno quando si riporta il moltiplicando in tabella (in figura)
  - **Moltiplicatore negativo:** si fa il complemento a 2 di Moltiplicando e Moltiplicatore per ritornare al caso di Moltiplicatore positivo



- Soluzione più generale ed efficiente:
  - **Algoritmo di Booth**

Algoritmo di Booth

- Nell'algoritmo di Booth si ricodifica il Moltiplicatore come somma e sottrazione di potenze di 2

- Caso semplice in cui il Moltiplicatore contiene una sequenza contigua di 1:

- **Q = 0011110 = 0100000 + 1111110 = 0100000 – 0000010 = $2^5 – 2^1$**

- **P = M * Q = M * 2^5 + M * -2^1 = M * 2^5 + -M * 2^1**

- Il prodotto è uguale al moltiplicando fatto scorrere di 5 posizioni a sinistra + il complemento a due del moltiplicando fatto scorrere di 1 posizione a sinistra

- Generalizzabile per qualsiasi Moltiplicatore:

- **Q = 1100111011 = 1100000000 + 0000111000 + 0000000011 = 0000000000 + 1100000000 + 0001000000 + 1111111000 + 0000000100 + 1111111111 = $-2^8 + 2^6 – 2^3 + 2^2 – 2^0$**

- **P = -M * 2^8 + M * 2^6 + -M * 2^3 + M * 2^2 + -M * 2^0**

Ricodifica bit-pair

- L'algoritmo di Booth può essere usato per ottenere una **ricodifica più efficiente**

- La **ricodifica bit-pair** raggruppa **in coppie i bit del moltiplicatore** ricodificato con Booth

- **Ciascuna coppia di bit** del moltiplicatore viene **riscritta come un singolo coefficiente** da moltiplicare al moltiplicando:

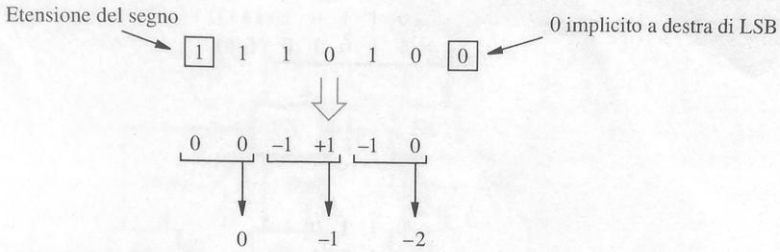
(+1, -1) → 2M – M → **1** * M → **(0, +1)**

(-1, +1) → -2M + M → **-1** * M → **(0, -1)**

(+1, 0) → 2M + 0M → **2** * M → **(0, +2)**

Ecc..

- In questo modo **il numero di addizioni** da eseguire durante la moltiplicazione **viene dimezzato**



(a) Esempio di ricodifica bit-pair derivata da ricodifica di Booth

Coppia di bit del moltiplicatore		Bit del moltiplicatore sulla destra	Versione del moltiplicando selezionata in posizione i
$i+1$	i	$i-1$	
0	0	0	$0 \times M$
0	0	1	$+1 \times M$
0	1	0	$+1 \times M$
0	1	1	$+2 \times M$
1	0	0	$-2 \times M$
1	0	1	$-1 \times M$
1	1	0	$-1 \times M$
1	1	1	$0 \times M$

(b) Tabella delle selezioni di versione del moltiplicando

Divisione di interi

- Anche per la divisione di interi prendiamo come esempio l'algoritmo imparato a scuola:

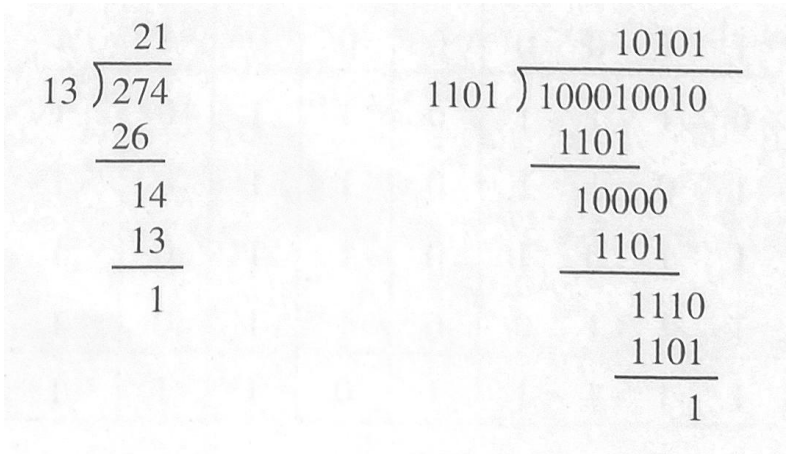
1. Si allinea il divisore con il dividendo a partire da sinistra

2. Si controlla se il divisore è contenuto nel dividendo e nel caso si calcola la divisione parziale tra le cifre allineate:

- si aggiunge il risultato al quoziente

- Si trascrive il resto in basso e gli si aggiunge la prossima cifra del dividendo

3. Si allinea il divisore e si riprende dal punto 2



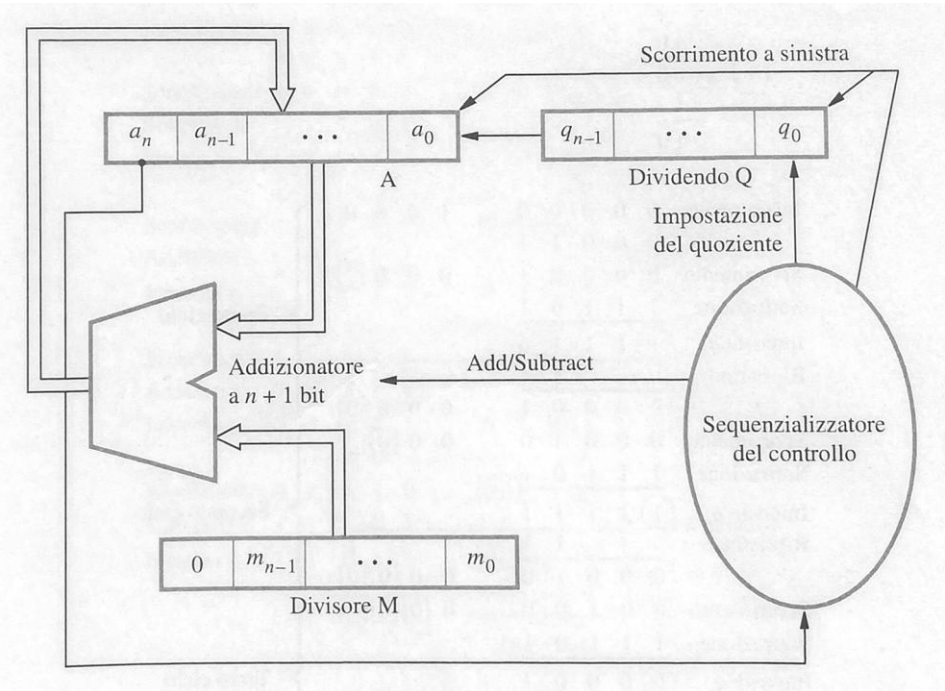
Divisione con ripristino

- Il circuito sequenziale per la divisione può essere realizzato con **un registro (M), 2 shift register (A e Q) e un addizionatore a n+1 bit**

- Eseguire n volte i seguenti 3 passi:
  - **Fare scorrere A e Q a sinistra di una posizione**
  - **Sottrarre M da A e porre il risultato in A**
  - **Se il segno di A è 1, porre q_0 a 0 e sommare M ad A; altrimenti, porre q_0 a 1**

- All'inizio **M contiene il Divisore, A contiene 0 e Q contiene il Dividendo**

- Alla fine **A contiene il Resto e Q contiene il Quoziente**



Divisione senza ripristino

- L'algoritmo di divisione può essere semplificato eliminando il passo di ripristino

- L'algoritmo senza ripristino si divide in due stadi:

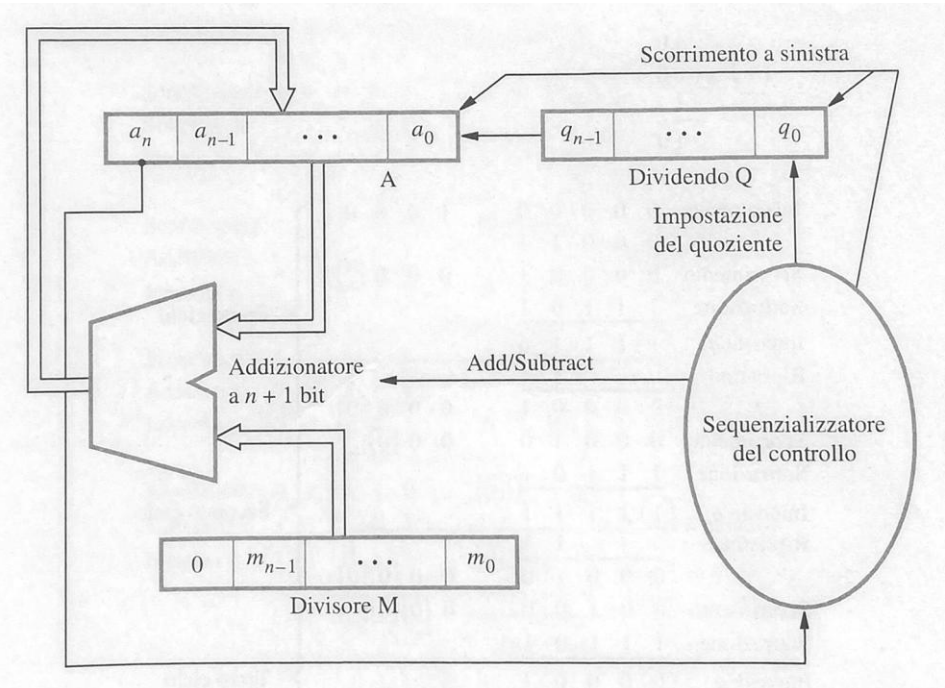
- **Stadio 1:** Eseguire n volte i seguenti 2 passi

- **Se il segno di A è 0, fare scorrere A e Q a sinistra di una posizione e sottrarre M da A; altrimenti, fare scorrere A e Q a sinistra di una posizione e sommare M ad A**

- **Se il segno di A è 0, porre q_0 a 1; altrimenti, porre q_0 a 0**

- **Stadio 2: passi:**

- **Se il segno di A è 1, sommare M ad A**



Numeri a virgola mobile

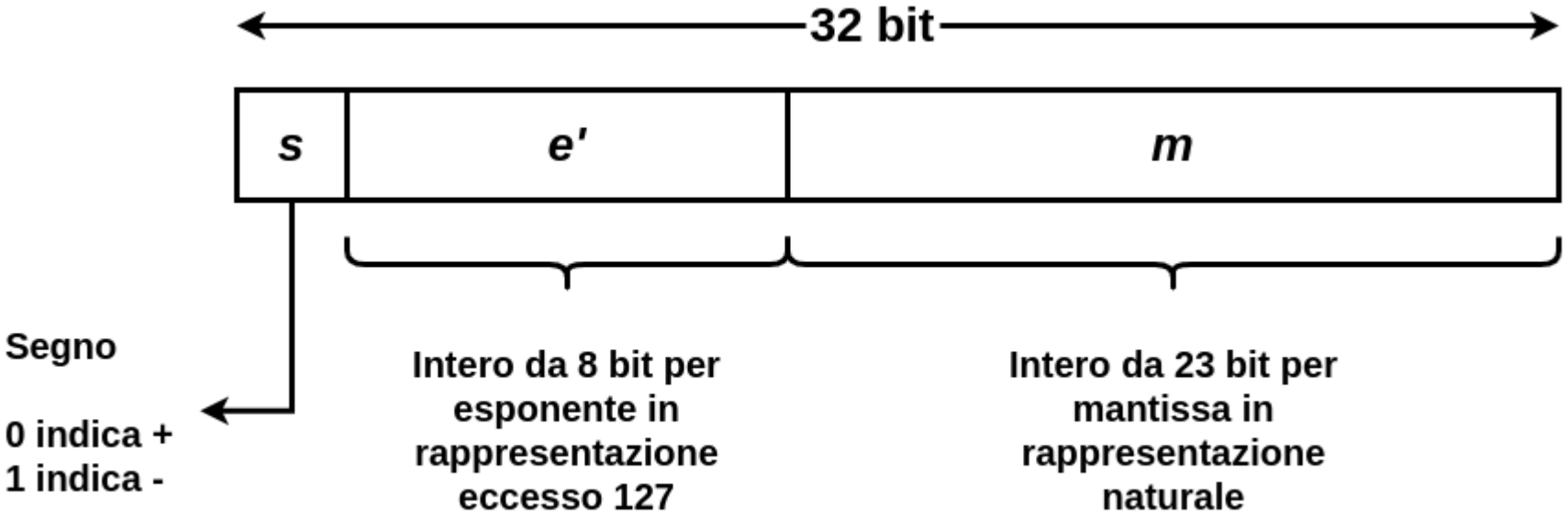
Un numero binario in virgola mobile può quindi essere rappresentato:

- Un **SEGNO** *s* per il numero
- La **MANTISSA** *m* (bit significativi escluso il bit più significativo)
- Un **ESPONENTE** *e* con segno in base 2

$$\text{Valore rappresentato} = \pm\, \mathbf{1, } \boldsymbol{m} \times \mathbf{2}^{\boldsymbol{e}}$$

Formato precisione singola (32 bit)

Standard **IEEE 754** numeri **32 bit**



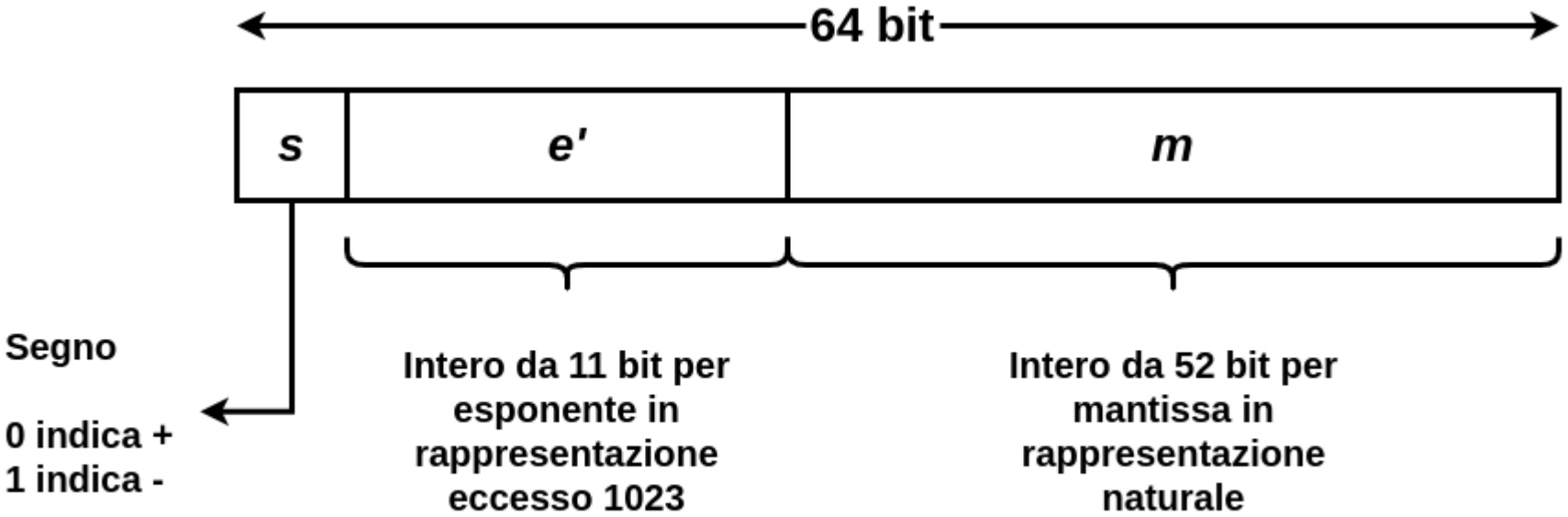
$0 \leq e' \leq 255$ $\quad e = e' - 127$

Valori speciali: $e' = 0$, $e' = 255$ Intervallo esponente: $-126 \leq e \leq 127$

Fattore di scala nell'intervallo: $[2^{-126}, 2^{127}]$

Formato precisione doppia (64 bit)

Standard **IEEE 754** numeri **64 bit**



$0 \leq e' \leq 2047$ $e = e' - 1023$

Valori speciali: $e' = 0$, $e' = 2047$ Intervallo esponente: $-1022 \leq e \leq 1023$

Fattore di scala nell'intervallo: $[2^{-1022}, 2^{1023}]$

Valori speciali

Alcuni valori dell'esponente sono speciali:

.*e′* = 0, *m* = 0 rappresenta lo **0 esatto**

.*e′* = 255(2047), *m* = 0 rappresenta l'infinito ∞

.*e′* = 0, *m* ≠ 0 rappresenta la **forma non normale**

.*e′* = 255(2047), *m* ≠ 0 rappresenta **Not a Number NaN**

Algoritmo di addizione/sottrazione algebrica

1. Scegli il numero con esponente più piccolo e fai scorrere la sua mantissa a destra di un numero di passi uguale alla differenza degli esponenti

2. Poni l'esponente del risultato uguale all'esponente più grande

3. Addiziona i fattori interi (1 || Mantissa) tenendo conto del segno

4. Se il numero risultante non è in forma normale, normalizzalo

Algoritmo di moltiplicazione algebrica

1. Addiziona gli esponenti dei fattori e togli 127 alla somma, ottenendo l'esponente provvisorio del prodotto

2. Moltiplica i fattori interi dei fattori ottenendo segno e fattore intero provvisorio del prodotto

3. Se il numero risultante non è in forma normale, normalizzalo

Algoritmo di divisione algebrica

1. Sottrai l'esponente del divisore dall'esponente del dividendo e aggiungi 127 alla differenza, ottenendo l'esponente provvisorio del rapporto

2. Calcola il quoziente del fattore intero del dividendo rispetto a quello del divisore, ottenendo segno e fattore intero provvisorio del rapporto

3. Se il numero risultante non è in forma normale, normalizzalo